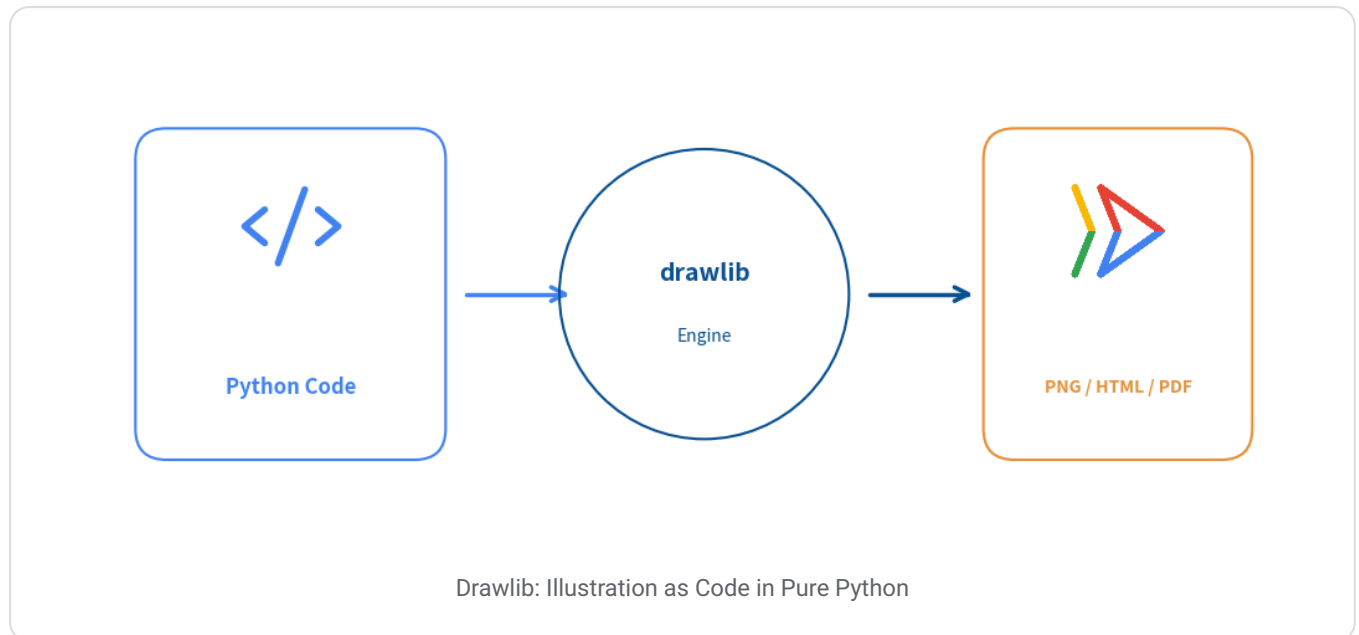


Drawlib Quickstart Guide

Illustration as Code & Documentation as Code in Pure Python

Drawlib is a modern Python library for creating technical diagrams, architecture blueprints, charts, SmartArts, and publication-ready documentation directly from code.



About This Document

This quickstart guide introduces the core workflows of **Drawlib**, from basic canvas coordinates and preset styling to typography, icons, external media, SmartArts, declarative charts, software diagrams, and the unified CLI document builder.

- **Repository:** <https://github.com/yuichil10/drawlib>
- **Author:** Yuichi Ito
- **License:** Apache License, Version 2.0

Table of Contents

1. About Drawlib

2. Installation & Setup

3. Canvas, Coordinates & Shapes

4. Preset Styles & Themes

5. Icons & External Images

6. Unified CLI & Document Builder

7. SmartArts Components

8. Declarative Charts

9. Software & Architecture Diagrams

1. About Drawlib

Drawlib is a pure Python drawing library designed to facilitate **Illustration as Code**. Instead of manually positioning shapes in GUI drawing tools that store opaque binary files, Drawlib generates crisp illustrations directly from readable, version-controlled Python scripts and Markdown documents.

Why Illustration as Code?

1. **Git & Code Review Friendly:** Every diagram is plain Python text. Track changes with `git diff`, review architecture updates in pull requests, and automate builds in CI/CD.
2. **Visual Consistency via Themes:** Apply unified preset styles (`default` , `google` , `monochrome` , or custom themes) across hundreds of illustrations with zero manual formatting.
3. **Full Power of Python:** Use loops, helper functions, data structures, and full IDE type hints / autocompletion instead of learning a restricted Domain-Specific Language (DSL).

First Example

Here is a simple example combining a styled circle and text label on a Drawlib canvas:

```
from drawlib.canvas import save, setup
from drawlib.shapes import circle
from drawlib.styles import Styles

setup(width=100, height=50)
circle(
    xy=(50, 25),
    radius=18,
    style=Styles.Primary,
    text="Hello drawlib!",
    textstyle=Styles.WhiteBold,
)
save()
```

When compiled by Drawlib, this code renders the following illustration:

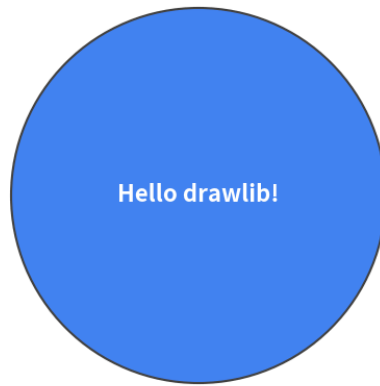


Figure 1.1: Styled Circle Generated from Python Code

2. Installation & Setup

Drawlib requires **Python 3.11 or higher** and can be installed from PyPI using `pip` or `uv`.

Installing with pip or uv

```
# Using pip
pip install drawlib

# Using uv
uv add drawlib
```

Verifying the Installation

Once installed, both the Python package `drawlib` and the `drawlib` command-line tool are available:

```
$ drawlib --version
```

You can also invoke the CLI via Python's module runner:

```
$ python -m drawlib --version
```

Semantic Versioning & Stability

Drawlib follows `<major>.<minor>.<patch>` semantic versioning:

- **Major Version:** Significant architectural or API changes
- **Minor Version:** New features and backward-compatible enhancements
- **Patch Version:** Bug fixes and internal optimizations without API changes



Figure 2.1: Drawlib Release & Versioning Workflow

3. Canvas, Coordinates & Shapes

Every Drawlib illustration takes place on a coordinate **Canvas**. By default, the canvas spans `width=100` and `height=100` coordinate units with the origin `(0, 0)` at the bottom-left corner.

Configuring the Canvas

Use `setup()` from `drawlib.canvas` to customize the coordinate dimensions, resolution (`dpi`), or alignment grid (`grid=True`):

```
from drawlib.canvas import setup

setup(width=100, height=50, dpi=200, grid=True)
```

- `width` / `height` : Logical coordinate space (default: `100` x `100`).
- `dpi` : Dots per inch resolution (default: `100` , where 10 inches x 100 DPI = 1000px width).
- `grid` : Overlays a coordinate grid and center axes to assist with precise placement.

Drawing Fundamental Primitives

Drawlib provides intuitive functions for lines, circles, rectangles, polygons, arrows, and text:

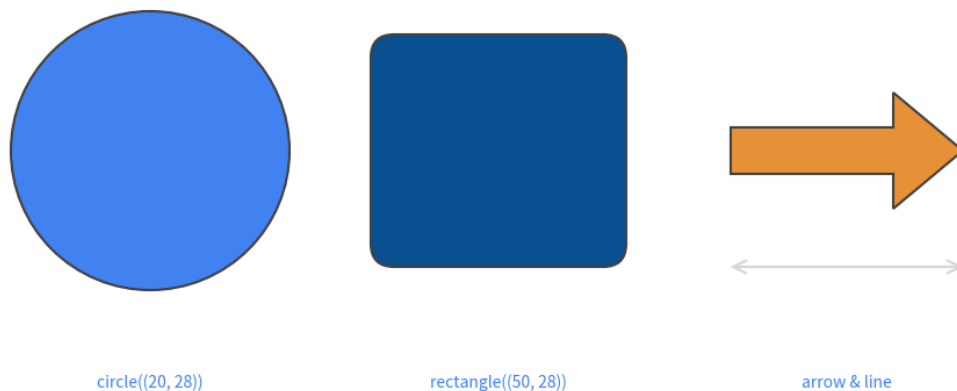


Figure 3.1: Fundamental Shapes and Coordinate Grid

Coordinate Alignment (`halign` and `valign`)

All shapes and text elements support horizontal (`left` , `center` , `right`) and vertical (`bottom` , `center` , `top`) alignment via `Style(halign=..., valign=...)` , making it effortless to align labels

and boxes around any anchor point (x, y) .

4. Preset Styles & Themes

Instead of manually configuring RGB tuples or custom `Style(...)` models for every shape, Drawlib includes a comprehensive **Preset Styles** system accessed via `from drawlib.styles import Styles`. Every drawing function accepts a preset style such as `style=Styles.Primary` or `style=Styles.DangerFlat`.

Official Style Themes

Drawlib ships with three curated preset themes:

- **"default"**: Balanced, clean palette designed for architectural blueprints and engineering diagrams.
- **"google"**: Modern, vibrant palette adhering to Google Cloud / Material design standards.
- **"monochrome"**: Crisp grayscale palette tailored for print books and academic publications.

Semantic Roles & Style Matrix

Drawlib defines 6 core semantic roles across all themes:

- **primary**: Main application logic, core services, and primary actions.
- **secondary**: Data stores, caches, queues, and auxiliary services.
- **accent**: Entry points, clients, gateways, and focal callouts.
- **muted**: Group containers, boundaries, subnets, and structural lines.
- **danger**: Errors, alerts, security vulnerabilities, and failures.
- **success**: Healthy status, completed milestones, and verified checks.

Each semantic role (and color) provides 10 orthogonal variants: `bordered` (default), `flat`, `outline`, `dashed`, `bold`, `light`, `outline_bold`, `outline_light`, `dashed_bold`, and `dashed_light`.

```
from drawlib.styles import Styles

circle((20, 25), radius=10, style=Styles.Primary)          # Primary bordered
circle((45, 25), radius=10, style=Styles.SecondaryFlat)    # Secondary flat
circle((70, 25), radius=10, style=Styles.DangerOutline)    # Danger outline
```

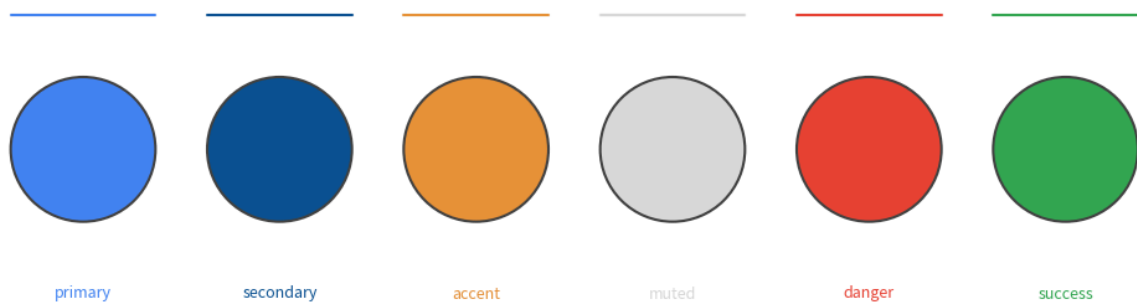


Figure 4.1: Core Semantic Styles Applied to Lines, Shapes, and Text

Typography & Custom Fonts

Drawlib includes universal CJK/Latin fonts ([Font](#)), sans-serif typography ([FontRoboto](#)), and code fonts ([FontMonoSpace](#)) via [drawlib.fonts](#) . You can also load custom font files using [FontFile](#) :

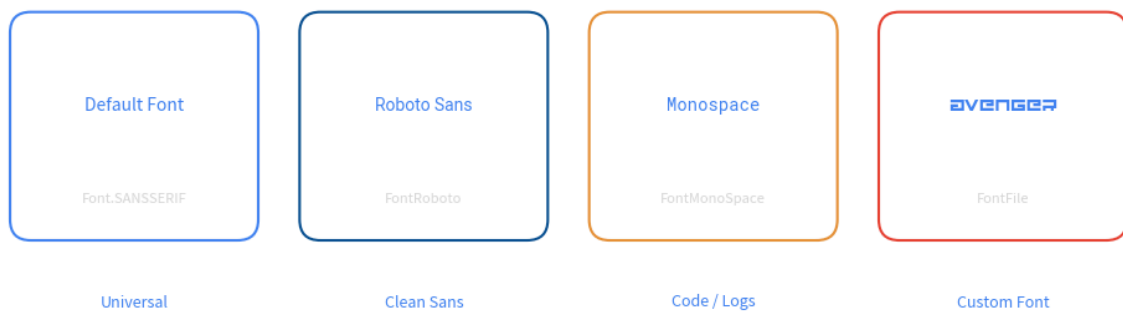


Figure 4.2: Typography and Custom Font Files

5. Icons & External Images

Drawlib bundles rich icon modules under `drawlib.icons` and provides seamless external image embedding with in-memory transformations via `drawlib.images`.

1. Phosphor Icons (`drawlib.icons.phosphor`)

`phosphor` provides ~1,500 scalable vector font icons covering devices, networking, users, arrows, and UI metaphors. Each icon is a Python function accepting `xy`, `width`, `angle`, and `style`:



`phosphor.desktop`



`phosphor.database`



`phosphor.cloud`



`phosphor.shield_check`

Figure 5.1: Vector Icons from `drawlib.icons.phosphor`

2. Google Cloud Platform Icons (`drawlib.icons.gcp`)

`drawlib.icons.gcp` provides 250+ official multi-color Google Cloud service and category icons designed for cloud architecture blueprints:



Load Balancing



Cloud Run



Cloud SQL



BigQuery

Figure 5.2: Official Google Cloud Service Icons from `drawlib.icons.gcp`

3. External Images & Media (`drawlib.images`)

For project logos, team avatars, or hardware badges, Drawlib provides the `image()` function and the `Dimage` utility class from `drawlib.images`. You can render raster images directly from file paths or apply chainable transformations (such as mirroring or sepia filtering):



`image('_assets/linux.png')`



`Dimage.mirror()`



`Dimage.sepia()`

Figure 5.3: External Image Embedding and Dimage Transformations

6. Unified CLI & Document Builder

Drawlib includes a built-in **Document Builder** and **CLI** (`drawlib`) that compiles standalone Python scripts or Markdown documentation containing embedded `drawlib` code blocks into multiple publication targets.

Core CLI Subcommands

Command	Description
<code>drawlib build image <src></code>	Execute <code>.py</code> files or directories to batch-generate illustration images.
<code>drawlib build markdown <src> -o <out></code>	Compile Markdown with <code>drawlib</code> blocks into standard GitHub-ready Markdown + images.
<code>drawlib build html <src> -o <out></code>	Compile Markdown/HTML into a responsive static HTML website (<code>--css google</code>).
<code>drawlib build pdf <src> -o <out.pdf></code>	Merge Markdown/HTML files in filename order into a single PDF (<code>--generate-index</code>).
<code>drawlib show <file> [block]</code>	Preview an illustration or Markdown block in desktop GUI viewer.
<code>drawlib show <file> [block] -o <img.png></code>	Export a specific <code>drawlib</code> block or script to an image file.
<code>drawlib serve <html_dir></code>	Start a local HTTP server with link and asset verification.

Example Build Commands

```
# Build GitHub Markdown and HTML website
uv run drawlib build markdown docs_src/ -o docs/
uv run drawlib build html docs_src/ -o docs_html/

# Build merged Quickstart PDF with auto-generated index between 1st and 2nd files
uv run drawlib build pdf quickstart_src/ -o quickstart.pdf --generate-index
```

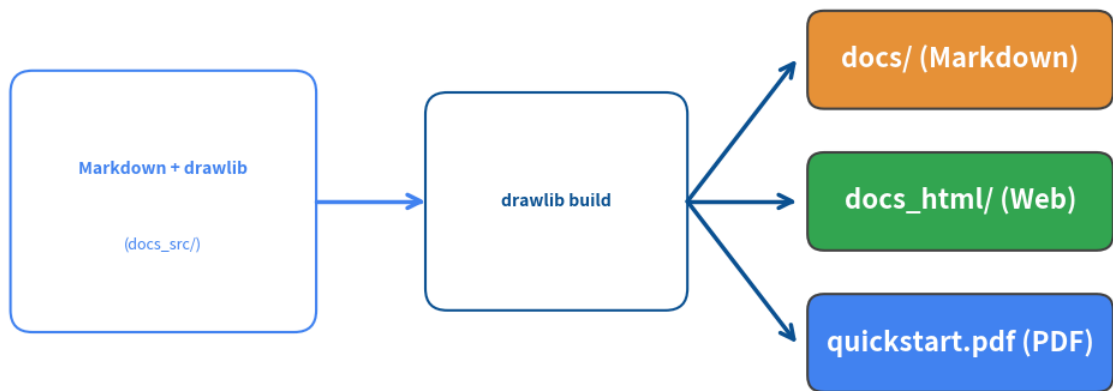


Figure 6.1: Single-Source Documentation Compilation Pipeline

7. SmartArts Components

`drawlib.smartarts` provides high-level presentation graphics inspired by PowerPoint SmartArt. Instead of calculating geometry manually, you instantiate a SmartArt component, add items, and call `.draw()`.

Available SmartArt Classes

- `Pyramid` : Hierarchical pyramid layers with customizable orientation and styles.
- `BoxList` / `ChevronProcess` : Sequential process blocks and arrowhead pipelines.
- `Cycle` / `MindMap` / `Tree` : Circular workflows, mind maps, and hierarchical trees.
- `Table` / `GridLayout` / `SourceCode` : Structured tables, grid cards, and syntax-highlighted code blocks.

Example: Hierarchical Pyramid

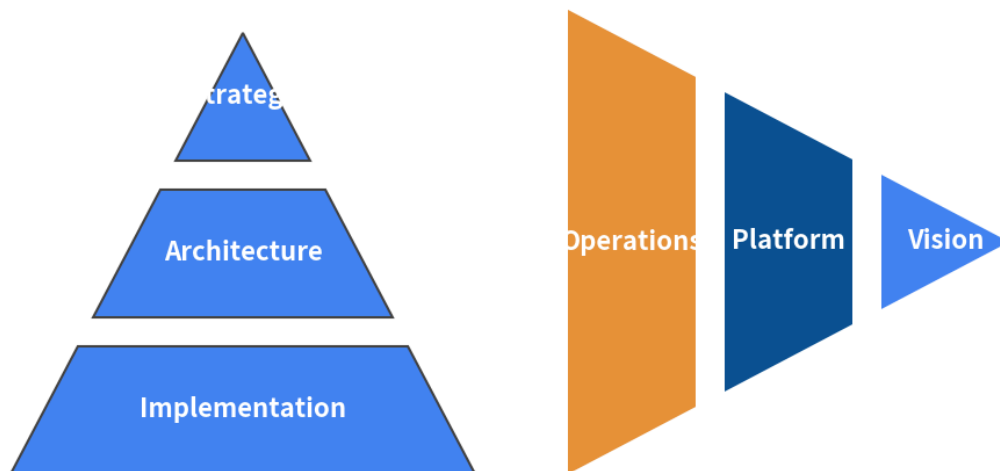


Figure 7.1: SmartArt Pyramid Comparing Default and Custom Styled Layers

8. Declarative Charts

`drawlib.charts` provides vector-grade data visualization built directly on Drawlib's canvas primitives. You can place charts alongside annotations, icons, or architecture diagrams on any canvas.

Supported Chart Types

- `BarChart` : Vertical and horizontal grouped/stacked bars with linear or logarithmic scales.
- `LineChart` & `AreaChart` : Trend lines, smooth splines, custom markers, and stacked areas.
- `PieChart` & `RadarChart` : Proportional slices, donut rings, and multivariate spider charts.
- `ScatterChart` & `GanttChart` : 2D correlation plots and project schedule timelines.

Example: Grouped Bar Chart

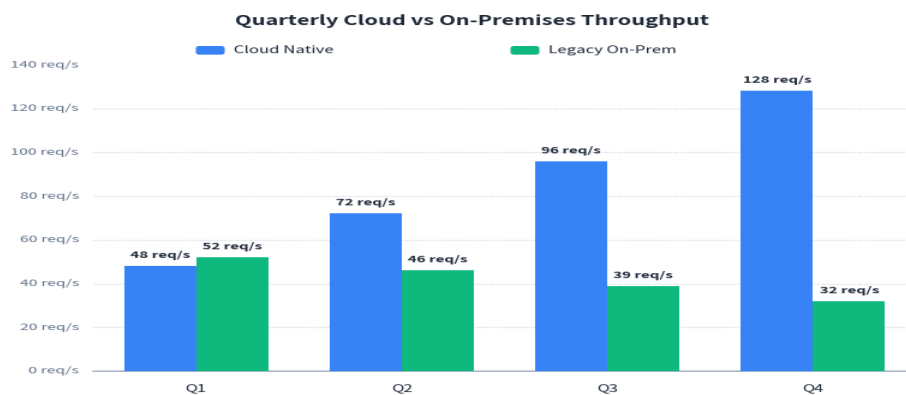


Figure 8.1: Declarative Grouped BarChart

9. Software & Architecture Diagrams

`drawlib.diagrams` provides specialized declarative engines for software engineering and cloud architecture documentation, including **Sequence Diagrams**, **Architecture Diagrams**, **Flow Diagrams**, **Class Diagrams**, **State Diagrams**, and **ER Diagrams**.

Sequence Diagrams (`drawlib.diagrams.sequence`)

With `SequenceDiagram`, participants and lifelines are defined as Python objects and interactions use expressive methods (`request()`, `reply()`) with automatic timeline spacing:

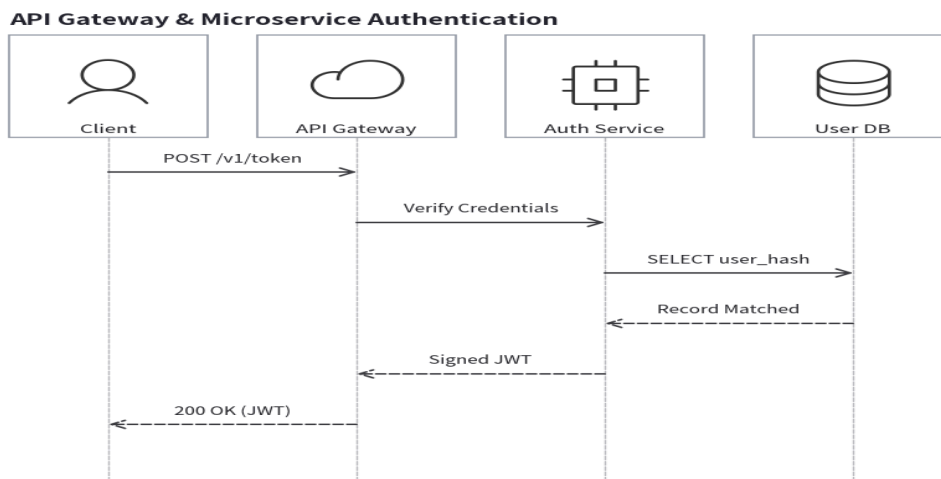


Figure 9.1: Declarative Sequence Diagram with Phosphor Icons